```
  it "allows us to complete a task" do
    task.mark_completed
    expect(task).to be_complete
  end
end
```

You have two specs here. The first says that a brand-new task is not complete. The second creates a different task object, marks it as complete, and expects that it is then complete.

This test is entirely at the API level and makes no claim about the underlying mechanism for representing completed tasks. This is great because it means you can change the implementation without breaking tests as long as the API still works. Overreliance on implementation details is a major cause of test fragility, so when you can describe the behavior rather than the implementation, you should do so.

> **Prescription 4** When possible, write your tests to describe your code's behavior, not its implementation.

You can make the test pass with some simple methods in Task:

basics/03/app/models/task.rb

```
class Task
  def initialize
    @completed = false
  end

  def mark_completed
    @completed = true
  end

  def complete?
    @completed
  end
end
```

This implementation probably won't survive long—tasks will probably grow more states—but for now it works.

> **Prescription 5** Keeping your code as simple as possible allows you to focus complexity on the areas that really need it.

You can use a similar test to ensure the project's ability to determine completeness: